

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №1
По предмету: “Вычислительная математика”
Вариант № 14**

Выполнил: Федоров Дмитрий
Александрович Р3206
Проверил: Зинченко А. А.
г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
1.	3
2.	3
3.	3
4.	4
Исходный код программы:	4
Результаты работы программы:	5
Заключение:	7

ЗАДАНИЕ:

Лабораторная работа №1. «Решение системы линейных алгебраических уравнений СЛАУ»

1. № варианта определяется как номер в списке группы согласно ИСУ.
2. Выбор языка программирования зависит от преподавателя-практика.
3. В программе численный метод должен быть реализован в виде отдельной подпрограммы/метода/класса, в который исходные/выходные данные передаются в качестве параметров.
4. Размерность матрицы $n \leq 20$ (задается из файла или с клавиатуры - по выбору конечного пользователя).
5. Должна быть реализована возможность ввода коэффициентов матрицы, как с клавиатуры, так и из файла (по выбору конечного пользователя).

Для итерационных методов должно быть реализовано:

- Точность задается с клавиатуры/файла,
- Проверка диагонального преобладания (в случае, если диагональное преобладание в исходной матрице отсутствует, сделать перестановку строк/столбцов до тех пор, пока преобладание не будет достигнуто). В случае невозможности достижения диагонального преобладания - выводить соответствующее сообщение.
- Вывод нормы матрицы (любой, на Ваш выбор),
- Вывод вектора неизвестных: $x_1, x_2, \dots, x_n$,
- Вывод количества итераций, за которое было найдено решение,
- Вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$.

Листинг программы:

```
def input_from_keyboard():
    print("Ввод с клавиатуры:")

    n = None
    while n is None:
        try:
            n = int(input("Введите размерность матрицы n <= 20 "))

            if (n <= 0) or (n > 20):
                print("n должно быть от 1 до 20")
                n = None

        except ValueError:
            print("Ошибка: необходимо ввести целое число!")
        except EOFError:
            print("Ошибка: неожиданный конец ввода")
            return None, None, None
```

```

epsilon = None
while epsilon is None:
    try:
        epsilon_input = input("Введите точность  $\epsilon$  например, 0.001: ")
        epsilon_input = epsilon_input.replace(',', '.')
        epsilon = float(epsilon_input)

        if epsilon <= 0:
            print("Ошибка: точность должна быть положительным числом!")
            epsilon = None
        elif epsilon > 1:
            print("Предупреждение: точность больше 1")

    except ValueError:
        print("необходимо ввести число")
    except EOFError:
        print("неожиданный конец ввода")
        return None, None, None

augmented_matrix = []

print("\nВведите расширенную матрицу [A|b]:")
print("Каждая строка должна содержать n коэффициентов A и один коэффициент b")

for i in range(n):
    row = None
    while row is None:
        try:
            user_input = input(f"Строка {i+1}: ")
            parts = user_input.strip().split()

            if len(parts) != (n + 1):
                print(f"ожидалось {n + 1} числа, получено {len(parts)}")
                continue

            row_values = []
            valid = True

            for j, part in enumerate(parts):
                try:
                    part = part.replace(',', '.')
                    value = float(part)
                    row_values.append(value)
                except ValueError:
                    print(f"Ошибка: '{part}' не является числом")
                    valid = False
                    break

            if valid:
                row = row_values

```

```

        except Exception as e:
            print(f"ошибка: {e}")

    augmented_matrix.append(row)

matrix_a = []
vector_b = []

for row in augmented_matrix:
    matrix_a.append(row[:n])
    vector_b.append(row[n])

return matrix_a, vector_b, epsilon

def input_from_file():
    print("Ввод из файла:")

    filename = input("Введите имя файла: ")

    try:
        with open(filename, 'r') as file:
            line = file.readline().strip()
            if not line:
                print("Ошибка: файл пуст")
                return None, None, None

    try:
        n = int(line)
    except ValueError:
        print("Первая строка должна содержать число")
        return None, None, None

    if n <= 0 or n > 20:
        print("Ошибка: размерность должна быть от 1 до 20")
        return None, None, None

    print(f"Размерность: n = {n}")

    line = file.readline().strip()
    if not line:
        print("Ошибка: в файле отсутствует точность  $\epsilon$ ")
        return None, None, None

    try:
        line = line.replace(',', '.')
        epsilon = float(line)
        if epsilon <= 0:
            print("Ошибка: точность должна быть положительным числом")
            return None, None, None
    except ValueError:

```

```

    print("Ошибка: точность должна быть числом")
    return None, None, None

print(f"Точность:  $\epsilon = \{\text{epsilon}\}$ ")

augmented_matrix = []

for i in range(n):
    line = file.readline()
    if not line:
        print(f"Ошибка: файл содержит только {i} строк, ожидалось {n}")
        return None, None, None

    parts = line.strip().split()

    if len(parts) != n + 1:
        print(f"Ошибка в строке {i+1}: ожидалось {n + 1} чисел")
        return None, None, None

    row = []
    valid = True

    for j, part in enumerate(parts):
        try:
            part = part.replace(',', '.')
            value = float(part)
            row.append(value)
        except ValueError:
            print(f"Ошибка в строке {i+1}: '{part}' не является числом")
            valid = False
            break

    if not valid:
        return None, None, None

    augmented_matrix.append(row)

matrix_a = []
vector_b = []

for row in augmented_matrix:
    matrix_a.append(row[:n])
    vector_b.append(row[n])

return matrix_a, vector_b, epsilon

except FileNotFoundError:
    print(f"Ошибка: файл '{filename}' не найден")
    return None, None, None

except Exception as e:
    print(f"Ошибка при чтении файла: {e}")
    return None, None, None

```

```

def print_system(matrix_a, vector_b, epsilon):
    if matrix_a is None or vector_b is None:
        return

    n = len(matrix_a)

    print("Расширенная матрица [A|b]:")

    for i in range(n):
        row_str = ""
        for j in range(n):
            row_str += f"{matrix_a[i][j]:10.3f} "
        row_str += f"| {vector_b[i]:10.3f}"
        print(row_str)

    print(f"Точность  $\varepsilon = \{epsilon\}$ ")

def has_diagonal_dominance(matrix):
    n = len(matrix)
    for i in range(n):
        diagonal = abs(matrix[i][i])
        sum_off_diagonal = 0
        for j in range(n):
            if j != i:
                sum_off_diagonal += abs(matrix[i][j])

        if diagonal < sum_off_diagonal:
            return False
    return True

def try_make_diagonal_dominance(matrix, b):
    n = len(matrix)

    new_matrix = [row[:] for row in matrix]
    new_b = b[:]

    used_rows = set()
    result_matrix = []
    result_b = []

    for i in range(n):
        best_row = -1
        max_ratio = -1

        for j in range(n):
            if j in used_rows:
                continue

            max_in_row = max(abs(x) for x in matrix[j])
            diagonal = abs(matrix[j][i])

```

```

    if diagonal < max_in_row * 0.9:
        continue

    sum_others = sum(abs(matrix[j][k]) for k in range(n) if k != i)
    if sum_others == 0:
        ratio = float('inf')
    else:
        ratio = diagonal / sum_others

    if ratio > max_ratio:
        max_ratio = ratio
        best_row = j

    if best_row == -1:
        print("Не удалось достичь диагонального преобладания перестановкой строк")
        return None, None

    result_matrix.append(matrix[best_row])
    result_b.append(b[best_row])
    used_rows.add(best_row)

    return result_matrix, result_b

def transform_to_c_form(matrix_a, vector_b):
    n = len(matrix_a)
    C = []
    d = []

    for i in range(n):
        if abs(matrix_a[i][i]) < 1e-10:
            print(f"Ошибка: нулевой диагональный элемент в строке {i+1}")
            return None, None

        row_C = [0.0] * n
        for j in range(n):
            if i != j:
                row_C[j] = -matrix_a[i][j] / matrix_a[i][i]

        C.append(row_C)
        d.append(vector_b[i] / matrix_a[i][i])

    return C, d

def calculate_matrix_norm(C):
    n = len(C)
    max_row_sum = 0

    for i in range(n):
        row_sum = sum(abs(C[i][j]) for j in range(n))
        max_row_sum = max(max_row_sum, row_sum)

```



```

return max_row_sum

def check_system_convergence(matrix_a, vector_b):

    if has_diagonal_dominance(matrix_a):
        print("Исходная матрица имеет диагональное преобладание")
        working_matrix = matrix_a
        working_b = vector_b
    else:
        print("Исходная матрица НЕ имеет диагонального преобладания")
        working_matrix, working_b = try_make_diagonal_dominance(matrix_a, vector_b)

    if working_matrix is None:
        print("Невозможно достичь диагонального преобладания")
        return None, None, None, False

    print("После перестановки строк диагональное преобладание достигнуто")
    print("\nПреобразованная система:")
    print_system(working_matrix, working_b, None)

    C, d = transform_to_c_form(working_matrix, working_b)

    if C is None:
        return None, None, None, False

    norm = calculate_matrix_norm(C)

    print(f"\nМатрица C:")
    for row in C:
        print(" " + " ".join(f"{x:8.3f}" for x in row))
    print(f"\nВектор d: {d}")
    print(f"\норма матрицы C: |C| = {norm:.3f}")

    if norm < 1:
        print(f"|C| = {norm:.3f} < 1 - условие сходимости выполняется")
    else:
        print(f"|C| = {norm:.3f} >= 1 - условие сходимости не выполняется")
        print("Метод может расходиться")

    return working_matrix, working_b, C, d, norm

def gauss_seidel_solve(C, d, epsilon, max_iterations=1000):
    n = len(d)

    x_old = d[:]
    x_new = [0.0] * n

    print(f"\нНачальное приближение: x^0 = {x_old}")

    iteration = 0

```

```

errors = []
all_iterations = [x_old[:]]

while iteration < max_iterations:
    iteration += 1

    for i in range(n):
        sum_Cx = 0
        for j in range(n):
            if j < i:
                sum_Cx += C[i][j] * x_new[j]
            elif j > i:
                sum_Cx += C[i][j] * x_old[j]

        x_new[i] = sum_Cx + d[i]

    current_errors = []
    max_error = 0
    for i in range(n):
        error = abs(x_new[i] - x_old[i])
        current_errors.append(error)
        if error > max_error:
            max_error = error

    errors.append(current_errors)
    all_iterations.append(x_new[:])

    print(f"\nИтерация {iteration}:")
    for i in range(n):
        print(f" x_{i+1}^{iteration} = {x_new[i]:.6f}")
    print(f" Погрешности: {[e:.6f].format(e) for e in current_errors}")
    print(f" Максимальная погрешность: {max_error:.6f}")

    if max_error <= epsilon:
        print(f"\n Достигнута требуемая точность за {iteration} итераций")
        break

    x_old = x_new[:]

if iteration >= max_iterations:
    print(f"\nДостигнуто максимальное число итераций ({max_iterations})")

return x_new, iteration, errors, all_iterations

def print_results(x, iterations, errors, C, d, epsilon):

    print("Результаты решения:")

    print(f"\nРешение найдено за {iterations} итераций")
    print(f"Точность  $\varepsilon$  = {epsilon}")

```

```

print("\nВектор неизвестных x:")
for i in range(len(x)):
    print(f" x_{i+1} = {x[i]:.8f}")

print("\nВектор погрешностей на последней итерации:")
for i in range(len(x)):
    print(f" |x_{i+1}^{iterations} - x_{i+1}^{iterations-1}| = {errors[-1][i]:.8f}")

def main():
    print("1/2 - (с клавиатуры/из файла)")

    choice = input("Ваш выбор (1 или 2): ")

    matrix_a = None
    vector_b = None
    epsilon = None

    if choice == '1':
        matrix_a, vector_b, epsilon = input_from_keyboard()
    elif choice == '2':
        matrix_a, vector_b, epsilon = input_from_file()
    else:
        print("Неверный выбор. Программа завершена.")
        return

    if matrix_a is not None and vector_b is not None and epsilon is not None:
        print_system(matrix_a, vector_b, epsilon)

        result = check_system_convergence(matrix_a, vector_b)

        if result[0] is not None:
            working_matrix, working_b, C, d, norm = result

            print("Начало итераций:")

            solution, iterations, errors, all_iterations = gauss_seidel_solve(C, d, epsilon)

            print("Итоговые результаты:")

            print(f"\nКоличество итераций: {iterations}")
            print(f"\nНорма матрицы C: |C| = {norm:.6f}")

            print("\nВектор неизвестных x:")
            for i in range(len(solution)):
                print(f" x_{i+1} = {solution[i]:.8f}")

            print("\nВектор погрешностей на последней итерации:")
            for i in range(len(solution)):
                print(f" |x_{i+1}^{iterations} - x_{i+1}^{iterations-1}| = {errors[-1][i]:.8f}")

```

```

print("погрешности:")

for i in range(len(matrix_a)):
    left = 0
    for j in range(len(matrix_a)):
        left += matrix_a[i][j] * solution[j]

    right = vector_b[i]
    diff = abs(left - right)
    print(f" Уравнение {i+1}: погрешность = {diff:.8f}")

else:
    print("\nСистема не подходит для решения")
else:
    print("\nНе удалось загрузить данные.")

if __name__ == "__main__":
    main()

```

Вывод

В данной лабораторной работе я изучил методы решения СЛАУ